Лекция 1

Эволюция технологий программирования

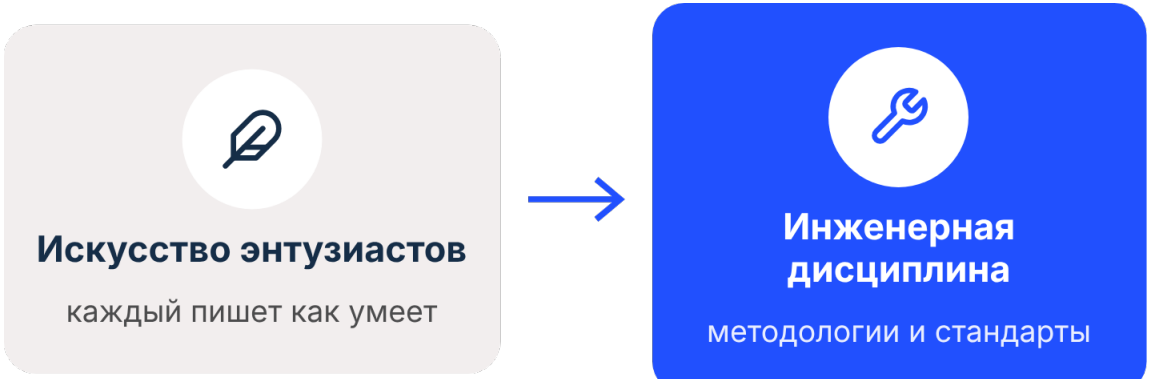
От машинных кодов и GOTO до микросервисов, облаков и ИИ



Зачем изучать эволюцию программирования

Разработка ПО, особенно в сфере ИБ, — сложный процесс, требующий системного подхода, дисциплины и проверенных методологий.

Программирование прошло путь **от искусства энтузиастов до инженерной дисциплины**

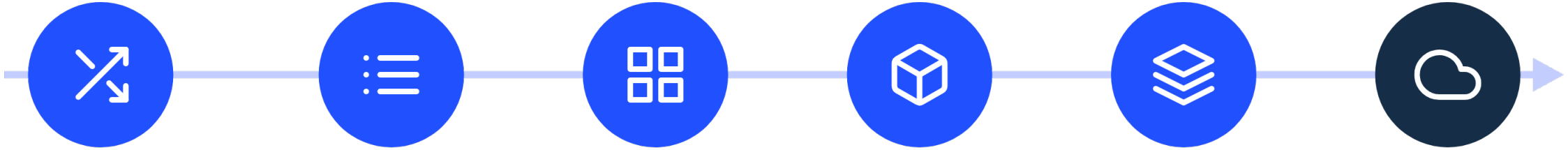


Что даёт понимание эволюции:

- **Осознанный выбор инструментов и подходов**
- **Предвидение будущих тенденций**
- **Оценка влияния на безопасность продукта**

Путь технологий программирования

Каждая новая парадигма не отменяла предыдущие, а вбирала их лучшие черты.



1950–60-е

Неструктурированное

машинные коды, GOTO

1960–70-е

Процедурное

Fortran, Algol, C, Pascal

1970-е

Модульное

интерфейс + реализация

1980–90-е

ООП

Smalltalk, C++, Java

с конца 1990-х

Компоненты и сервисы

COM, .NET, микросервисы

сегодня

Облака, DevOps, ИИ

CI/CD, генерация кода

Декларативное программирование развивается параллельно: функциональное и логическое

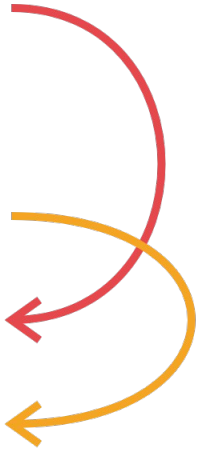
Неструктурированное программирование

1950–1960-е: машинные коды и ассемблеры. Главное — экономия памяти и вычислительных ресурсов.

Логика держится на переходах **GOTO**

Чтобы понять программу, нужно прыгать по строкам

```
10: ВВОД A
20: ВВОД B
30: ЕСЛИ A > B ПЕРЕЙТИ К 60
40: ВЫВОД B
50: ПЕРЕЙТИ К 70
60: ВЫВОД A
70: КОНЕЦ
```

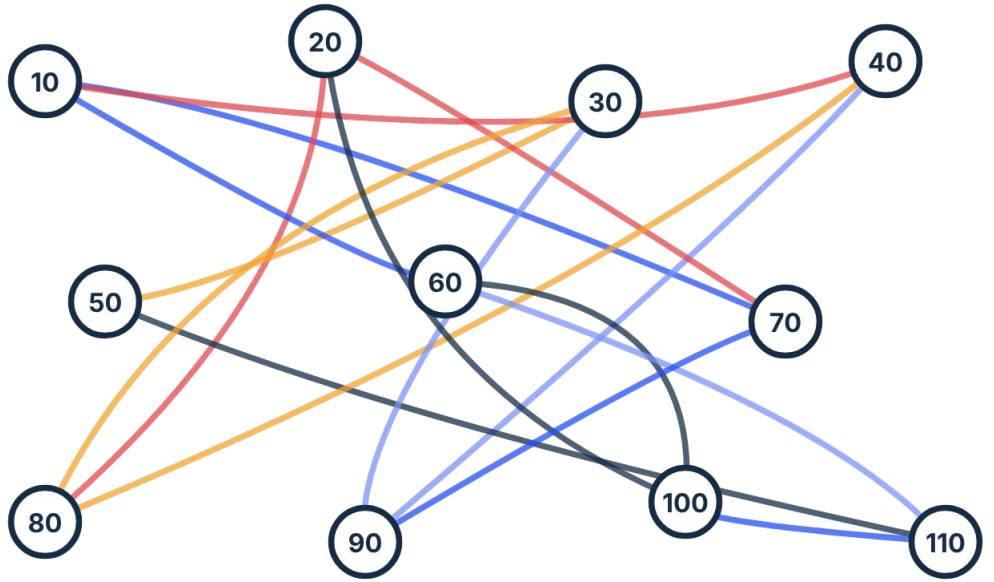


«Спагетти-код»

Запутанный клубок безусловных и условных переходов.

Источник огромного количества ошибок

Разработка крупных систем — невероятно сложна



Переходы
порядок постоянно меняется

Нет абстракций
всё в одном файле

Отладка
изменение → непредсказуемо

Процедурное программирование

1960–1970-е: языки высокого уровня и отказ от безусловных переходов.

Ключевая идея: программа — набор процедур и функций, каждая решает одну задачу

Эдсгер Дейкстра, 1968

«Go To Statement Considered Harmful»

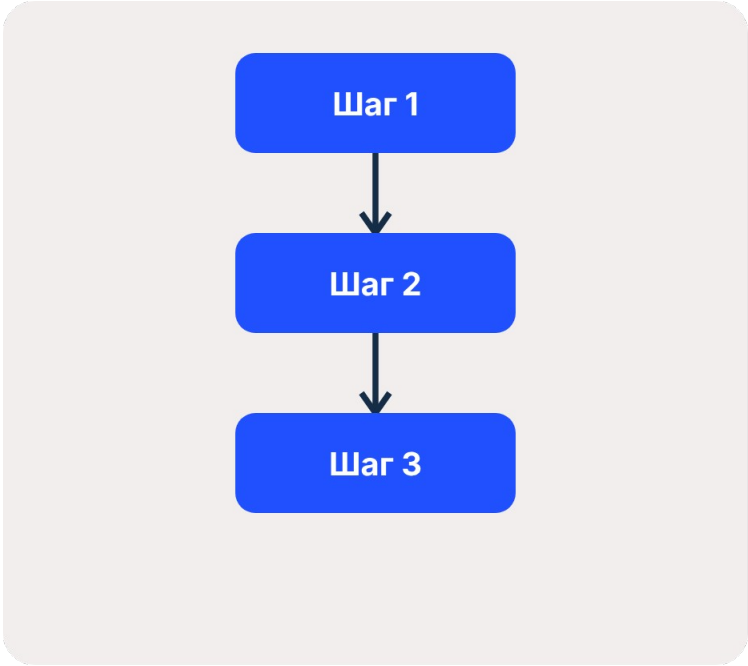
~~GOTO~~ отказ от безусловных переходов

Языки высокого уровня:

- Fortran
- Algol
- C
- Pascal

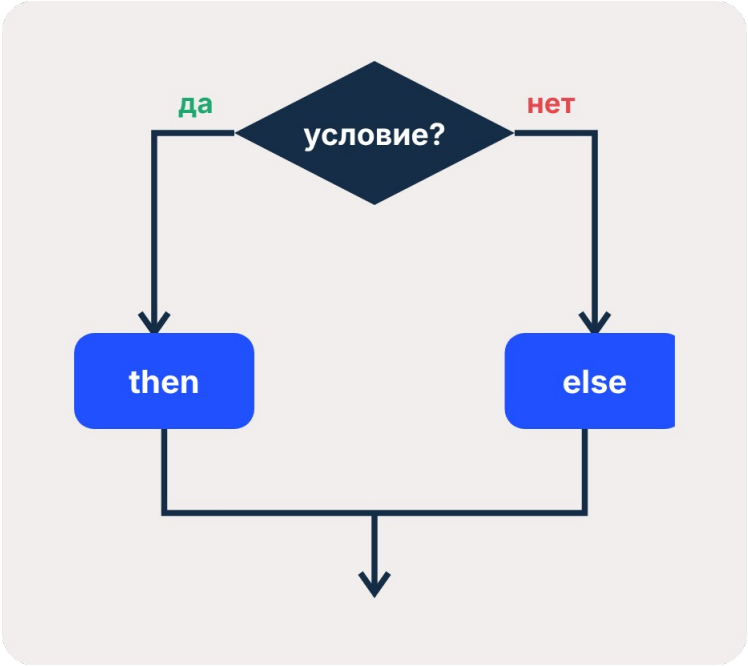
Структурное программирование: три конструкции

Любой алгоритм можно записать только с помощью трёх базовых управляющих конструкций.



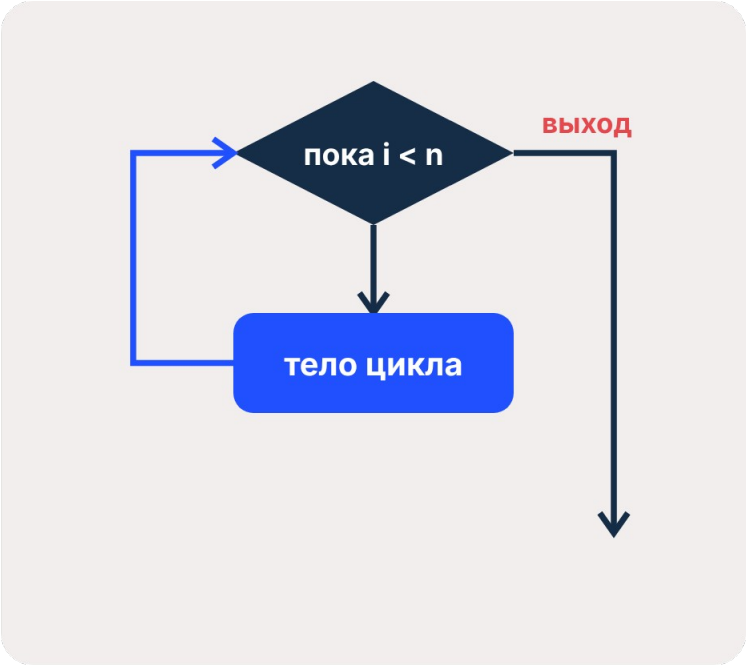
Последовательность

шаги выполняются по порядку



Ветвление

if-then-else



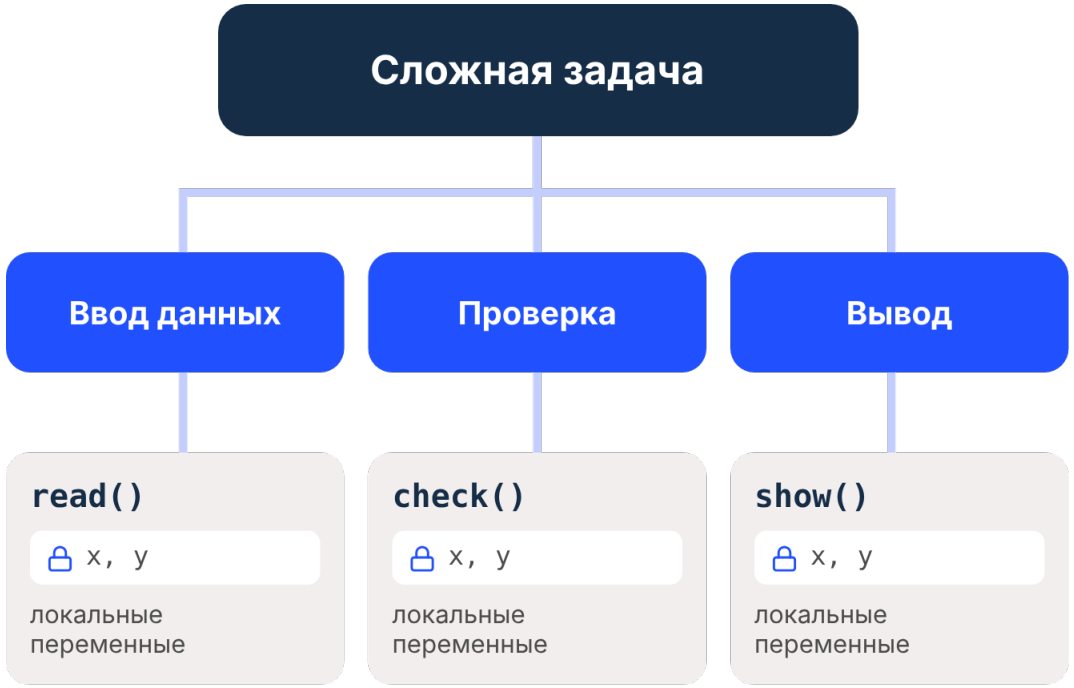
Цикл

while, for

Декомпозиция и локальные переменные

Сложная задача разбивается на простые подзадачи — каждая становится отдельной процедурой.

Данные внутри процедуры **изолированы** от остальной программы



Подзадача → процедура. Данные внутри изолированы

Процедурный подход: плюсы и минусы

Код стал понятнее, но с ростом программы появились новые проблемы.

Преимущества

Понятность
код логичен и читаем

Тестируемость
процедуру проверить проще

Повторное использование
одна процедура — много вызовов

Недостатки

Запутанные связи
процедур становится всё больше

Глобальные данные
меняются непредсказуемо

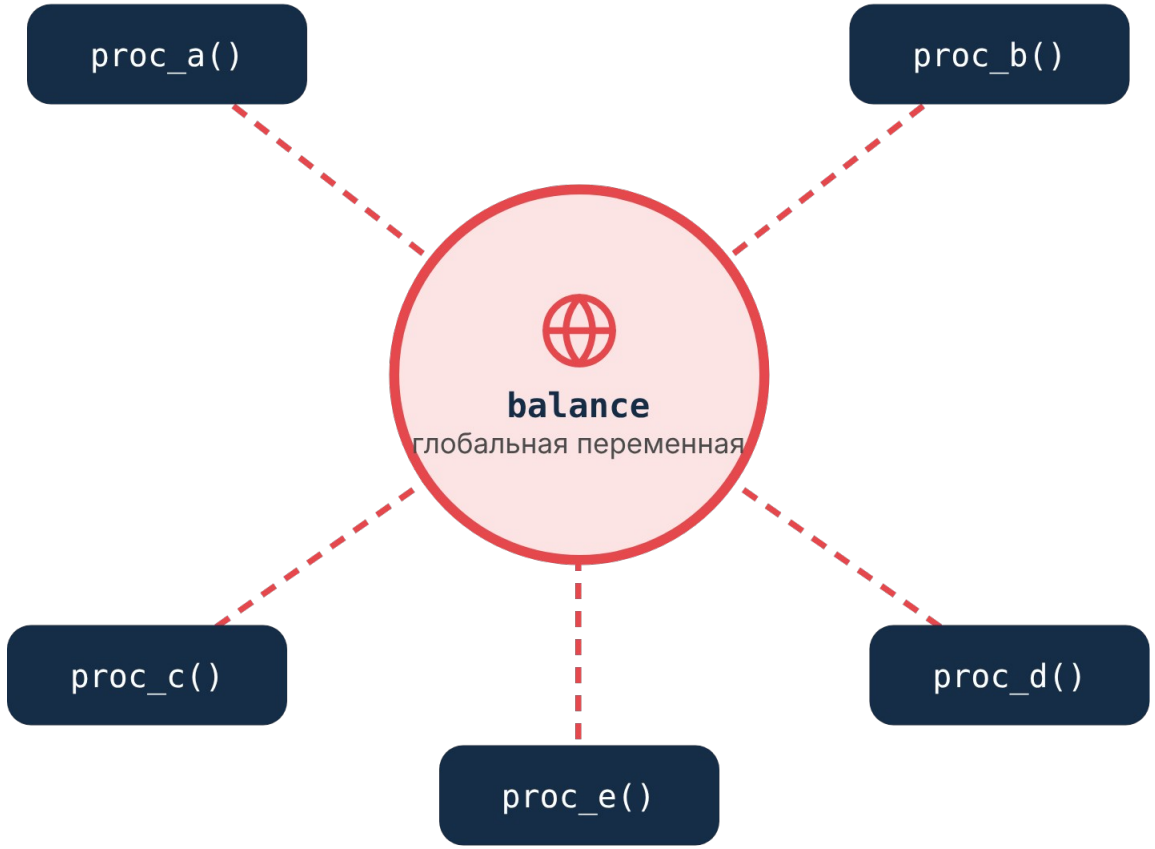
Сложная отладка
трудно найти, кто что изменил

Проблема глобальных данных

Многие процедуры имеют доступ к одним и тем же глобальным данным.

Данные могут **непредсказуемо изменяться**

Трудно найти, какая процедура их испортила

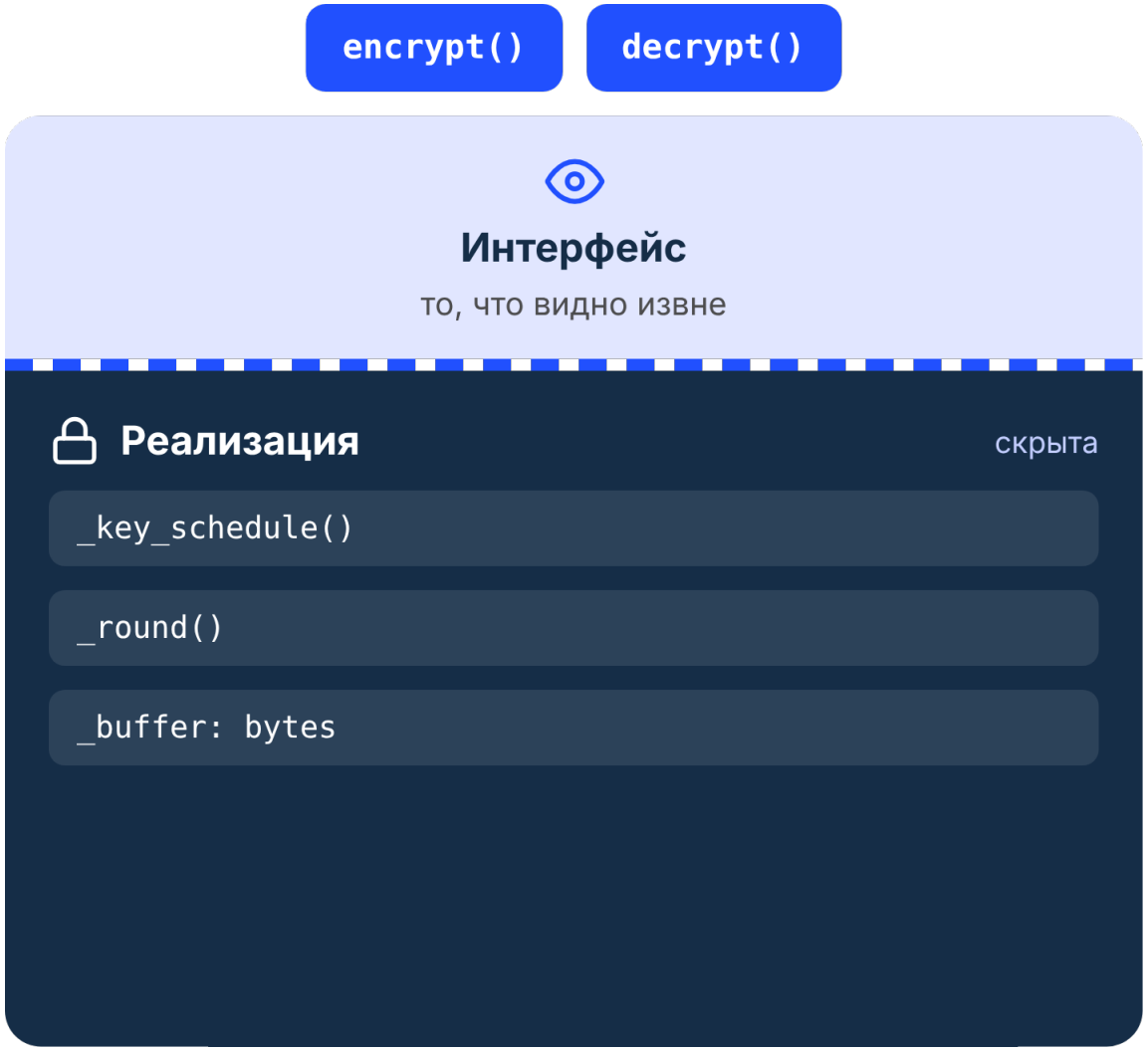


Модульное программирование

Модуль — логически завершённая часть программы с интерфейсом и реализацией.

Интерфейс — то, что видно извне

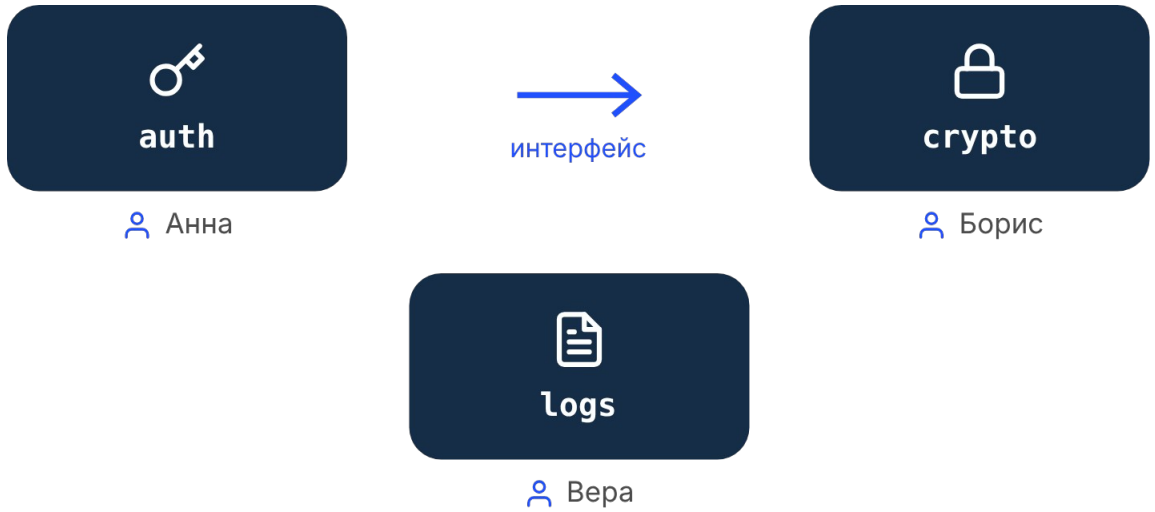
Реализация — то, что скрыто



Модули и интерфейсы

Программа состоит из независимых модулей, которые общаются через чётко определённые интерфейсы.

Модульный подход стал **фундаментом ООП**



Инкапсуляция данных

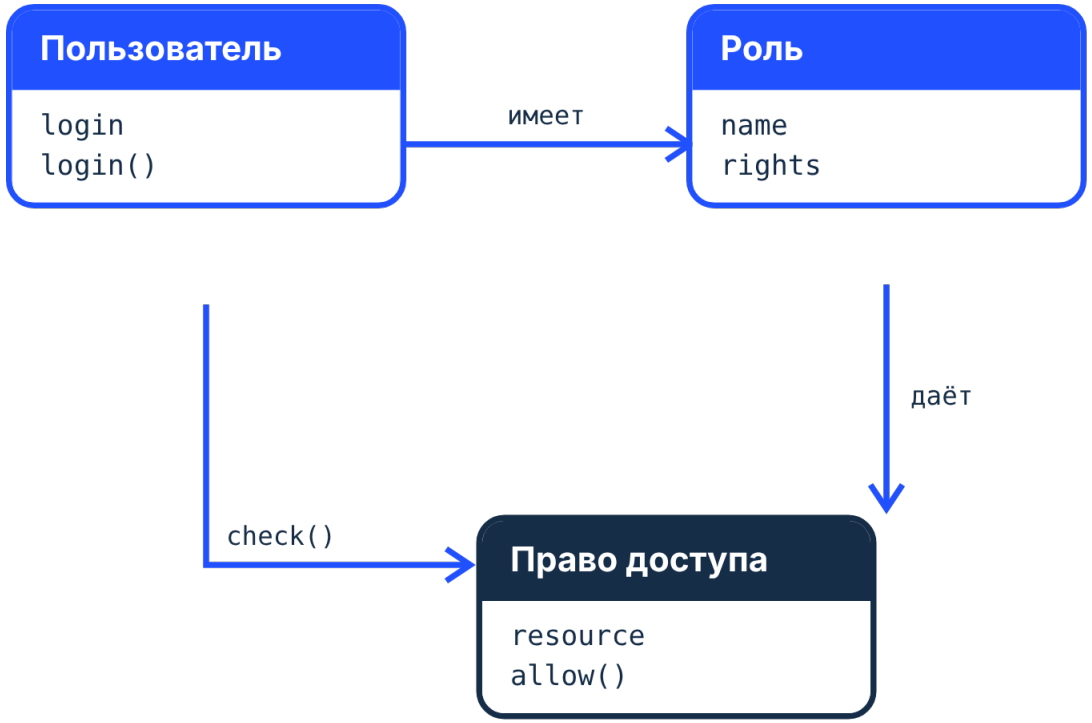
Ошибка не ломает всё

Параллельная работа

Объектно-ориентированное программирование

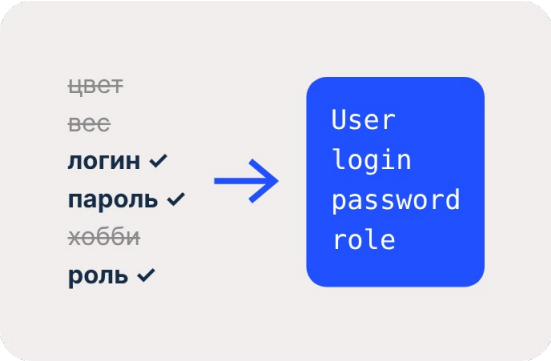
1980–1990-е: программа — совокупность взаимодействующих объектов.

На десятилетия стало доминирующей парадигмой



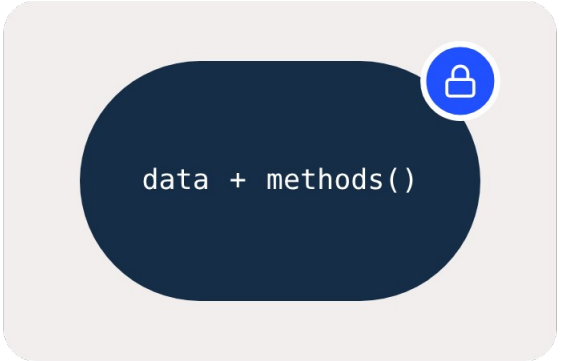
Smalltalk C++ Java C#

Четыре концепции ООП



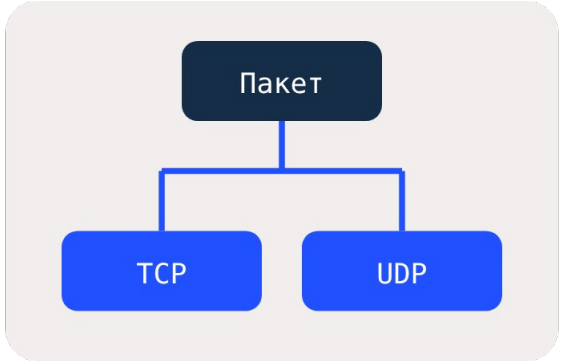
Абстракция

только существенные характеристики



Инкапсуляция

данные и методы в одном классе, реализация скрыта



Наследование

новый класс на основе существующего



Полиморфизм

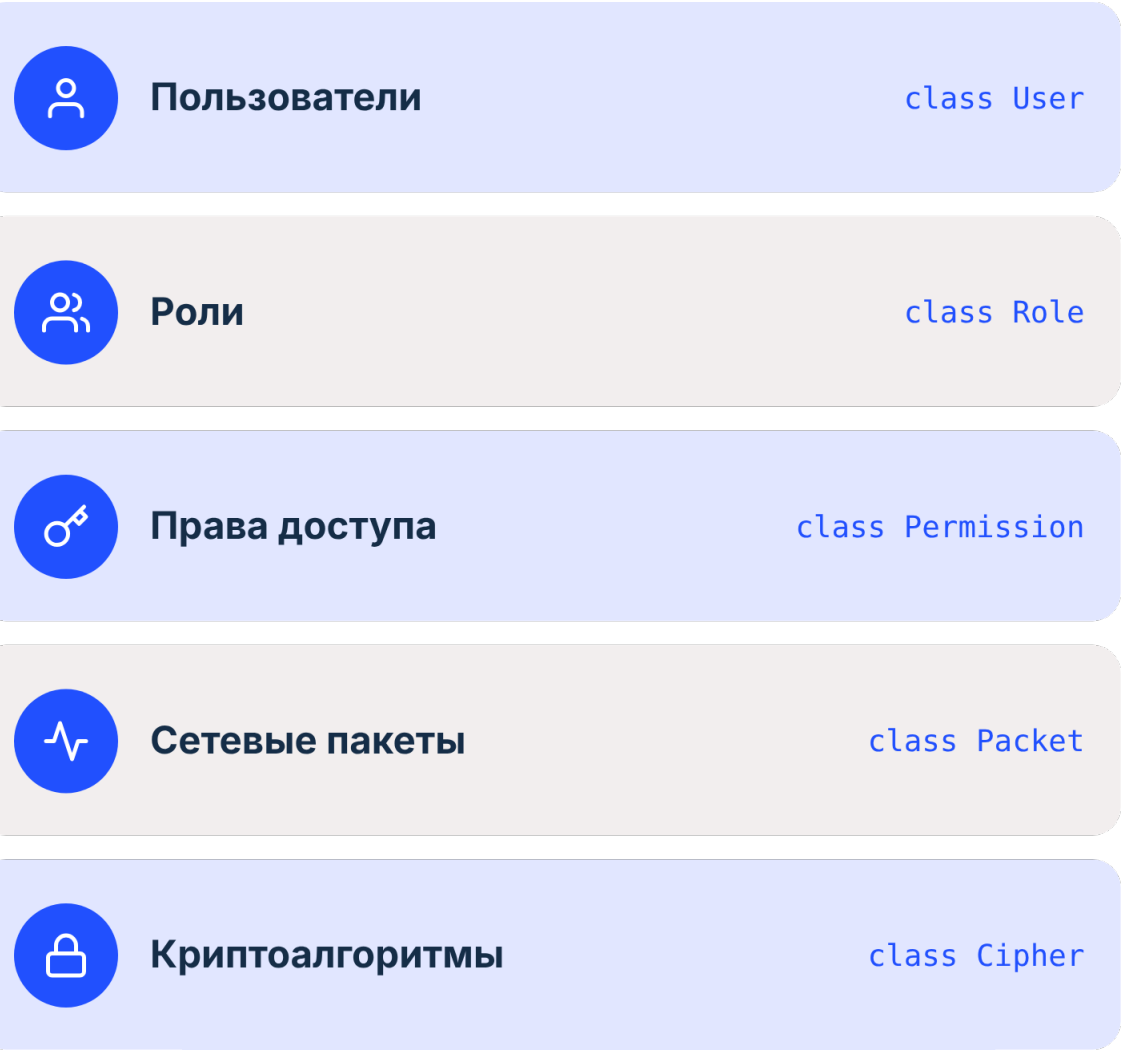
одинаковый интерфейс, разная реализация

ООП в информационной безопасности

ООП позволяет создавать гибкие, масштабируемые и расширяемые системы.

Код моделирует предметную область

Незаменимо при моделировании сложных сущностей



Императивно и декларативно

Императивный подход описывает **последовательность действий**, декларативный — **что нужно получить**.

Пример: найти имена всех администраторов

Императивно — КАК
последовательность действий

```
result = []
for u in users:
    if u.role == "admin":
        result.append(u.name)
```

Декларативно — ЧТО
описание результата

```
SELECT name FROM users
WHERE role = 'admin'
```

Функциональное программирование

Программа — вычисление значений функций.

Важны **предсказуемость** и отсутствие побочных эффектов



тот же вход → всегда тот же выход, без побочных эффектов

Чистые функции

Неизменяемость

Функции высшего порядка

Haskell · Lisp · Erlang · Python · JavaScript · Scala

В ИБ: парсеры протоколов, анализаторы трафика

Логическое программирование

Программа — набор фактов и правил.
Решение система ищет сама.

Язык **Prolog**

Экспертные системы и искусственный интеллект

Факты

```
admin(ivanov).
user(petrov).
```

Правила

```
access(X) :-
  admin(X).
```

Система сама ищет решение

```
?- access(ivanov).
true.
?- access(petrov).
false.
```

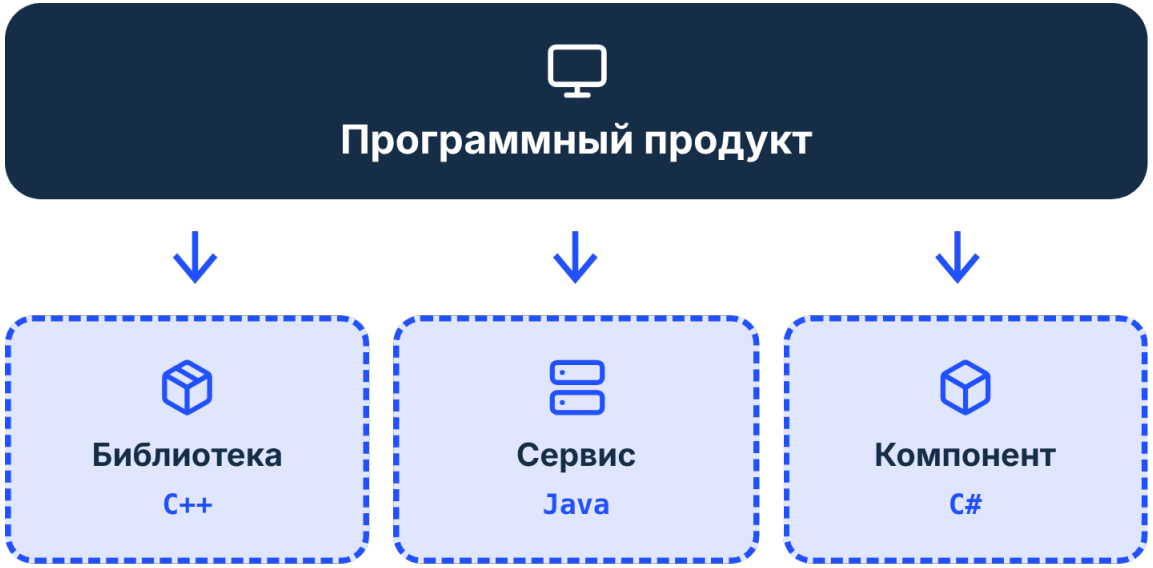
Prolog | экспертные системы | ИИ

Компонентный подход

С конца 1990-х ПО собирают из независимых, взаимозаменяемых компонентов.

Библиотеки и сервисы на разных языках и платформах

Основа для **микросервисной архитектуры**



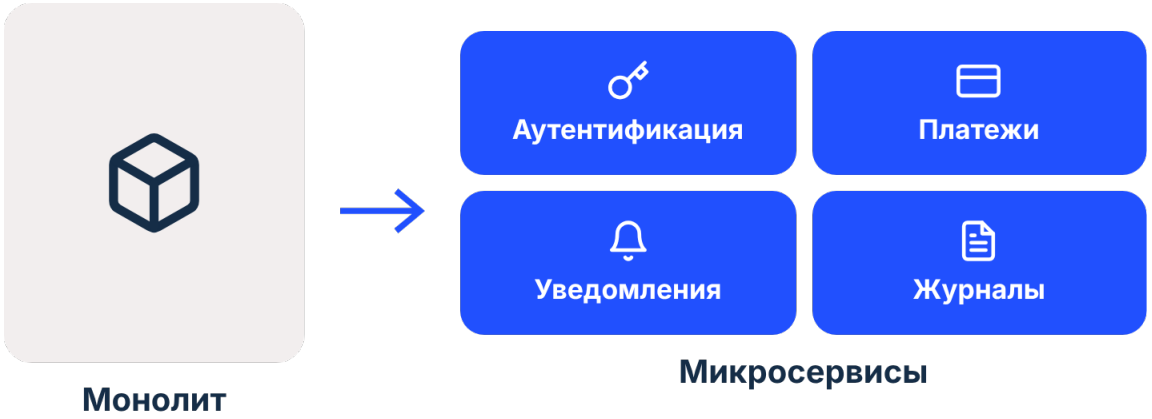
независимые, взаимозаменяемые, на разных языках и платформах

COM CORBA .NET JavaBeans

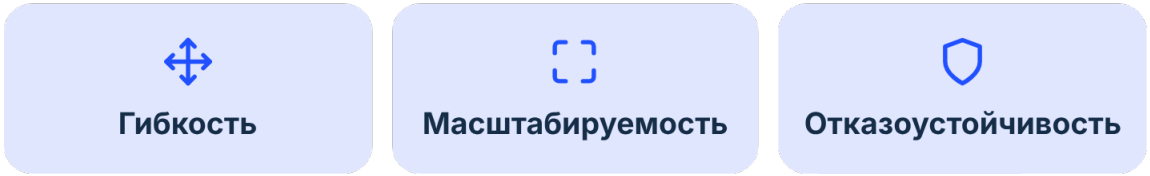
SOA и микросервисы

Приложение разбивается на множество небольших, слабо связанных сервисов.

Каждый сервис реализует **одну бизнес-функцию**

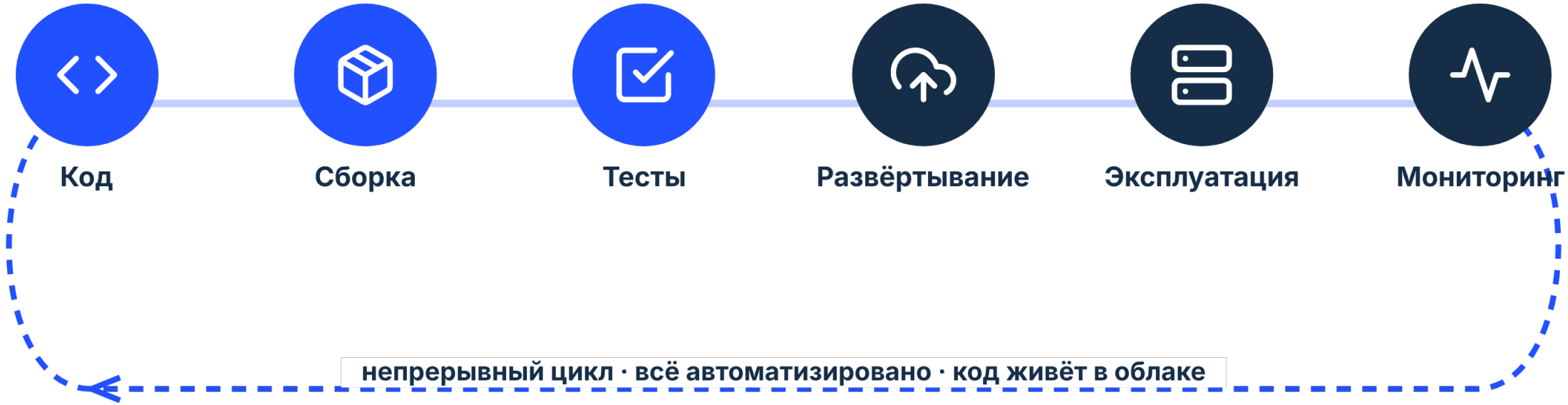


каждый сервис — одна бизнес-функция, связи слабые



Облачные вычисления и DevOps

Разработка и эксплуатация становятся единым непрерывным процессом — **CI/CD**.

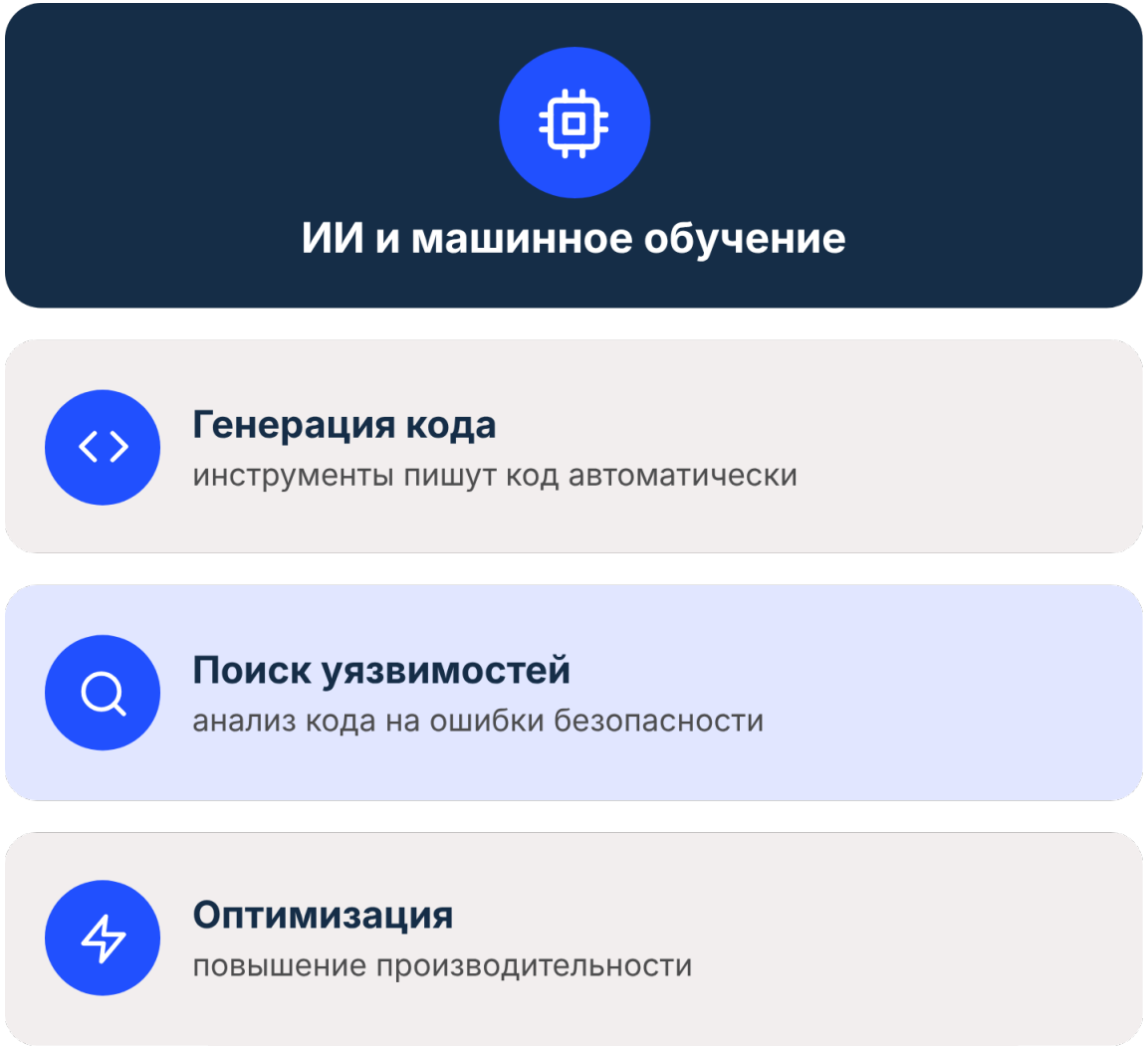


Dev — разработка **Ops** — эксплуатация

Искусственный интеллект

ИИ всё сильнее влияет на технологии программирования.

Инструменты автоматически генерируют код, находят уязвимости и оптимизируют производительность



Заключение

Эволюция программирования — путь от хаоса к порядку, от ремесла к инженерии.

Специалисту по ИБ важно понимать, **какая парадигма лежит в основе фреймворка**, её сильные и слабые стороны и влияние на безопасность

